

**UNIVERSITÀ DEGLI STUDI DI SALERNO**  
DIPARTIMENTO DI INFORMATICA

## **Ingegneria del Software**

UniClass - Requirement Analysis Document

v2.0



DATA: 22/02/2026

|                    |                                                                                                                          |
|--------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Scritto da:</b> | Giuseppe Sabetta (GS), Sara Gallo (SG), Saverio D'Avanzo (SD), Gerardo Antonio Cetrulo (AC), Lucageneroso Cammarota (LC) |
|--------------------|--------------------------------------------------------------------------------------------------------------------------|

## Revision History

| Data       | Versione | Descrizione                                                                                                                                 | Autore         |
|------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| 14/09/2026 | 2.0      | Aggiunta di requisiti di sicurezza, affidabilità e manutenibilità attraverso framework ASVS, SAMM, CR e Configuration Management Guidelines | GS             |
| 22/02/2026 | 1.9      | Aggiunta dei casi d'uso e consistenza tra UC e SD models                                                                                    | GS             |
| 20/02/2026 | 1.8      | Collasso di FR non più necessari (21-27) e refactoring di quelli esistenti                                                                  | LC             |
| 20/02/2026 | 1.8      | Modifica a definizioni e acronimi                                                                                                           | LC             |
| 30/01/2025 | 1.7      | Revisione e modifiche della formattazione e leggibilità                                                                                     | GS, SG, DC, AC |
| 29/01/2025 | 1.6      | Modifica dei casi d'uso e correzioni alla formattazione                                                                                     | GS, SG, SD, AC |
| 29/12/2024 | 1.5      | Aggiunta di Casi d'uso e varie correzioni alla formattazione                                                                                | GS, SG, SD, AC |
| 09/12/2024 | 1.4      | Modifica dei Sequence Diagram e casi d'uso per consistenza                                                                                  | GS, SG, SD, AC |
| 11/11/2024 | 1.3      | Modifica StateChart Diagrams e casi d'uso aggiunta lezione                                                                                  | GS, SG, SD, AC |
| 10/11/2024 | 1.2      | Aggiunta Object e Dynamic Modeling                                                                                                          | GS, SG, SD, AC |
| 08/11/2024 | 1.1      | Aggiunta, modifiche e rimozione dei requisiti funzionali e non funzionali                                                                   | GS, SG, SD, AC |
| 27/10/2024 | 1.0      | Requirements Analysis Document (RAD)                                                                                                        | GS, SG, SD, AC |

# Indice

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                       | <b>4</b>  |
| 1.1      | Purpose of the system . . . . .                           | 4         |
| 1.2      | Scope del Sistema . . . . .                               | 4         |
| 1.3      | Objectives and success criteria for the project . . . . . | 4         |
| <b>2</b> | <b>Sistema Attuale</b>                                    | <b>6</b>  |
| <b>3</b> | <b>Proposed System</b>                                    | <b>6</b>  |
| 3.1      | Overview . . . . .                                        | 6         |
| 3.2      | Requisiti Funzionali . . . . .                            | 6         |
| 3.3      | Requisiti Funzionali Associati ai Ruoli . . . . .         | 7         |
| 3.3.1    | Funzionalità Didattiche e di Pianificazione . . . . .     | 7         |
| 3.3.2    | Comunicazione e Messaggistica . . . . .                   | 7         |
| 3.3.3    | Amministrazione del Sistema . . . . .                     | 8         |
| 3.3.4    | Prenotazioni e Questionari . . . . .                      | 8         |
| 3.4      | Requisiti non Funzionali . . . . .                        | 8         |
| 3.5      | Modelli di Sistema . . . . .                              | 16        |
| 3.5.1    | Scenari . . . . .                                         | 16        |
| 3.5.2    | Use Case Model . . . . .                                  | 19        |
| 3.5.3    | Object Model . . . . .                                    | 25        |
| 3.5.4    | Dynamic Model . . . . .                                   | 27        |
| <b>4</b> | <b>AI-Usage</b>                                           | <b>32</b> |

# 1 Introduzione

## 1.1 Purpose of the system

### Introduzione

UniClass è un ambiente digitale open-source facilmente accessibile a tutti, in grado di rispondere a ogni esigenza.

Questo applicativo non si limita a migliorare l'esperienza degli studenti universitari, ma semplifica anche le interazioni dei docenti con il sistema e con gli studenti stessi. UniClass permette la reperibilità in tempo reale di informazioni sulla quotidianità degli studenti e del personale in ateneo, come visualizzazione degli orari delle lezioni, calendarizzazione di quest'ultime e garantire interoperabilità tra lo studente e il docente per semplificare le loro interazioni e l'andamento dei corsi.

### Descrizione del Problema

Le funzionalità offerte dagli attuali strumenti sono spesso troppo simili e si sovrappongono tra loro. Ad esempio, per sapere se un'aula è libera o occupata, uno studente deve utilizzare più applicazioni differenti su più piattaforme. Non solo le applicazioni sono dispersive e molto simili tra loro, creando confusione negli utenti, ma gli studenti, nella maggior parte dei casi, si trovano in difficoltà per la mancata calendarizzazione dei corsi da seguire: uno studente che deve seguire un corso del secondo anno e, subito dopo, del terzo sarà costretto a sprecare troppo tempo in una ricerca che diventerà frustrante e ripetitiva.

## 1.2 Scope del Sistema

Questo sistema è rivolto principalmente agli studenti universitari e al personale docente. Esistono diverse tipologie di utenti per le interazioni con il sistema o con altri tipi di utenti o semplicemente per la gestione delle risorse digitali disponibili in piattaforma riguardanti l'organizzazione d'ateneo.

Ciò di cui non si occuperà UniClass sarà la gestione finanziaria, l'iscrizione dello studente e gestioni amministrative riguardanti i ruoli del personale universitario.

## 1.3 Objectives and success criteria for the project

### Obiettivi:

- Fornire una piattaforma centralizzata per visualizzare le aule libere e gli orari delle lezioni.
- Migliorare la navigazione del campus tramite mappe dettagliate.
- Garantire comunicazione tra studente e docente
- Ridurre il tempo e lo sforzo necessari per trovare informazioni rilevanti.

### Benefici:

- Miglioramento dell'efficienza e della produttività degli studenti e del personale docente.
- Riduzione dello stress associato alla ricerca di aule e orari.
- Miglioramento della gestione degli spazi universitari

### Definizioni, acronimi e abbreviazioni

**Utente:** Qualsiasi soggetto (registrato o meno) che interagisce con la piattaforma UniClass. Ogni Utente registrato possiede un profilo univoco identificato da email e credenziali.

**Ruolo:** Attributo dinamico associato a un Utente registrato che ne determina le funzionalità accessibili e le responsabilità all'interno del sistema. Un Utente può possedere uno o più ruoli contemporaneamente (es. un dottorando può avere contemporaneamente il ruolo di Studente e di Docente). I ruoli principali previsti sono:

- **Studente:** Permette l'accesso ai servizi didattici, alla prenotazione appelli e alla fruizione dei corsi.
- **Docente:** Permette la gestione della didattica, l'invio di comunicazioni broadcast e l'interazione con gli studenti.
- **Coordinatore di Corso:** Estende i privilegi del Docente con funzionalità di gestione e coordinamento del corso di laurea.
- **Personale Tecnico-Amministrativo (PTA):** Permette la gestione degli account, il supporto tecnico e l'amministrazione della piattaforma.

**Ospite:** Utente non autenticato che ha a disposizione un numero limitato di funzioni come la visualizzazione dei corsi, degli orari e delle aule libere.

**Utente Autenticato:** Qualsiasi Utente che ha eseguito l'accesso. Le funzionalità disponibili sono determinate dal Ruolo (o dai Ruoli) posseduti.

## 2 Sistema Attuale

I sistemi attualmente in uso dagli utenti dell'Università degli Studi di Salerno sono UNISA Lezioni, MyUniSalerno ed Easycourse. I tre sistemi sono rispettivamente due applicazioni disponibili per Android e iOS ed una piattaforma web.

Esistono delle funzionalità troppo simili tra loro presenti in più applicazioni e creano disagio all'utente. La dispersione e distribuzione delle operazioni su più piattaforme rallenta la reperibilità di informazioni riguardanti le lezioni e i corsi da seguire. UNISA Lezioni, in particolare, è utilizzato principalmente per la visualizzazione delle aule libere che, però, non garantisce un'interfaccia grafica abbastanza intuitiva tale da rispondere rapidamente alle esigenze dello studente, rappresentando banalmente un cerchio verde o rosso raffigurante la disponibilità dell'aula, senza mostrare, però, gli orari, che sono molto più importanti.

Per di più, UNISA Lezioni garantisce la visualizzazione degli orari degli esami che, però, sono gestiti anche in MyUniSalerno.

La pecca principale è che le aule libere sono visualizzabili su UNISA Lezioni ma gli orari delle lezioni sono disponibili solo su Easycourse.

La mancanza di centralizzazione delle funzionalità operazionali a lungo andare stufa l'utente, non permettendogli di sfruttare a pieno i vari servizi offerti dai sistemi attuali.

## 3 Proposed System

### 3.1 Overview

Il nostro sistema ha come obiettivo la centralizzazione delle funzionalità all'interno di una sola piattaforma web, in grado di migliorare l'interoperabilità tra più tipi di utenti e garantire un'ottima esperienza universitaria attraverso i servizi resi disponibili su UniClass.

### 3.2 Requisiti Funzionali

#### Requisiti Funzionali dell'Ospite

- **RF1** - Visualizzazione della pagina principale del sito web  
Priorità di implementazione: **Alta**
- **RF2** - Visualizzare gli orari di un singolo corso di uno specifico corso di Laurea.  
Priorità di implementazione: **Alta**
- **RF3** - Ricercare l'orario settimanale di un corso di Laurea.  
Priorità di implementazione: **Alta**
- **RF4** - Visualizzare le aule libere di un determinato edificio  
Priorità di implementazione: **Alta**
- **RF5** - Visualizzare la mappa d'Ateneo.  
Priorità di implementazione: **Bassa**

- **RF6** - Visualizzare le informazioni sull'Applicazione  
Priorità di implementazione: **Alta**

### Requisiti Funzionali dell'Utente

- **RF7** – Autenticarsi  
Priorità di implementazione: **Alta**
- **RF8** – Può visualizzare le informazioni del suo account come Corso di Laurea, informazioni anagrafiche, matricola, e-mail istituzionale  
Priorità di implementazione: **Alta**

## 3.3 Requisiti Funzionali Associati ai Ruoli

Poiché il sistema UniClass adotta un modello basato su Ruoli dinamici, i requisiti funzionali sono qui categorizzati in base alle capacità operative abilitate dal possesso di uno specifico Ruolo. Un singolo Utente può soddisfare i requisiti di più categorie se possiede ruoli multipli.

### 3.3.1 Funzionalità Didattiche e di Pianificazione

- **RF10 - Gestione Orari e Lezioni:** Gli utenti con ruolo **Coordinatore** possono creare, modificare ed eliminare lezioni per il corso di laurea di competenza. Devono garantire che non vi siano sovrapposizioni di aule o docenti.  
*Priorità di implementazione: Media*
- **RF11 - Gestione Appelli:** Gli utenti con ruolo **Docente** possono visualizzare gli studenti prenotati agli appelli dei propri corsi e consultare i risultati dei questionari di valutazione.  
*Priorità di implementazione: Bassa*
- **RF12 - Agenda Personale:** Gli utenti con ruolo **Studente** possono creare e visualizzare un'agenda personale contenente gli orari dei corsi seguiti.  
*Priorità di implementazione: Bassa*

### 3.3.2 Comunicazione e Messaggistica

- **RF13 - Messaggistica Punto-Punto:** Gli utenti con ruolo **Studente** o **Docente** possono comunicare direttamente tra loro tramite messaggistica privata.  
*Priorità di implementazione: Alta*
- **RF14 - Comunicazione Broadcast (Docente):** Gli utenti con ruolo **Docente** possono inviare avvisi broadcast a tutti gli studenti iscritti ai propri corsi.  
*Priorità di implementazione: Alta*
- **RF15 - Comunicazione Broadcast (Coordinatore):** Gli utenti con ruolo **Coordinatore** possono inviare comunicazioni broadcast a tutti gli studenti del corso di laurea coordinato.  
*Priorità di implementazione: Media*

- **RF16 - Ricezione Comunicazioni:** Tutti gli utenti autenticati possono ricevere e visualizzare comunicazioni e avvisi.  
*Priorità di implementazione: Alta*

### 3.3.3 Amministrazione del Sistema

- **RF17 - Gestione Account:** Gli utenti con ruolo **PersonaleTecnicoAmministrativo** possono creare nuovi account UniClass, assegnando loro i Ruoli appropriati, e rimuovere account esistenti.  
*Priorità di implementazione: Bassa*
- **RF18 - Attivazione Utenti:** Gli utenti con ruolo **PersonaleTecnicoAmministrativo** gestiscono l'attivazione iniziale degli account e il recupero delle credenziali.  
*Priorità di implementazione: Bassa*

### 3.3.4 Prenotazioni e Questionari

- **RF19 - Prenotazione Esami:** Gli utenti con ruolo **Studente** possono prenotarsi agli appelli disponibili.  
*Priorità di implementazione: Bassa*
- **RF20 - Compilazione Questionari:** Gli utenti con ruolo **Studente** possono rispondere ai questionari di valutazione per le lezioni seguite.  
*Priorità di implementazione: Bassa*

## 3.4 Requisiti non Funzionali

### Usability

- **NFR1:** La piattaforma web deve presentare un alto contrasto dei colori tra i vari elementi grafici ed una dimensione del testo maggiore o uguale a 16px per rendere più accessibile e leggibile il sito. - **Priorità Alta**
- **NFR2:** La piattaforma web deve essere responsive. - **Priorità Alta**

### Reliability

- **NFR3:** In caso di input sbagliato nella schermata di autenticazione, Il sistema mostrerà all'utente la formattazione giusta dei parametri del Log In. - **Priorità Alta**
- **NFR4:** Aggiunta di un self-healing system rule-based per la messa in sicurezza delle funzionalità più critiche della piattaforma - **Priorità Bassa**

### Performance

- **NFR5:** L'accesso ad una pagina non deve richiedere più di 3 secondi. - **Priorità Alta**
- **NFR6:** La piattaforma deve supportare una connessione simultanea ad essa di minimo 20 utenti. - **Priorità Alta**

## Supportability

- **NFR7:** Il sistema deve essere scalabile per permettere facilmente possibili aggiunte di funzionalità. **Priorità Medio-Alta**
- **NFR8:** Aggiunta dei git updates come configuration items, uso di comment conventions per definire il completamento di task imposti nel RAD e tags per definire Pre e Post Manutenzione - **Priorità Alta**
- **NFR9:** Ogni aggiornamento della codebase deve essere definito con titolo e commento esplicativo con eventuali identificativi utili ai test di sicurezza e requisiti ASVS per acceptance testing successivo - **Priorità Alta**
- **NFR10:** Dopo ogni update è necessario effettuare una code review di tipo manuale (o ibrido comprendendo l'uso dell'IA ma solo di supporto, non essendo completamente efficace) ed eventualmente designare fix per le problematiche riscontrate subito dopo - **Priorità Alta**
- **NFR11:** Gli sviluppatori devono seguire le linee guida ufficiali (e.g. Angular commit cheat sheet) per la scrittura di git commit comprensibili. - **Priorità Alta**
- **NFR12:** Sono necessari Architectural Design Records (ADR) per eventuali cambiamenti strutturali importanti per descrizione, identificativo e rationale, e Secure Design Record (SDR) per eventuali cambiamenti nella codebase o nell'architettura software per l'aspetto sicuro dell'ambiente, descrivendo anche qui il Rationale. Questo sarà importante per Defect Management - **Priorità Alta**

## Security

La sezione attuale precede la sezione degli ASVS per via di un cambio del contesto applicativo, del dominio di conoscenza o di altri aspetti fondamentali non determinati dal primo livello del framework citato.

- **NFR13:** Abilitazione di un sistema basato sull'intelligenza artificiale per la visione di vulnerabilità in tempo reale presenti nelle dipendenze (3rd party libraries) del sistema UniClass - **Priorità Medio-Alta**
- **NFR14:** Abilitazione di una Security Policy all'interno della Master Repository GitHub di UniClass - **Priorità Alta**
- **NFR15:** Aggiunta di un sistema AI-based per la visione delle vulnerabilità di sicurezza presenti nel sistema UniClass - **Priorità Media**
- **NFR16:** Integrazione CI/CD per il monitoring in tempo reale dello stato sicuro di UniClass per ogni aggiornamento nella sua Master Repository - **Priorità Medio-Alta**
- **NFR17:** Integrazione con SonarQube per le pipeline CI/CD
- **NFR18:** Aggiunta di meccanismi bloccanti in base alle vulnerabilità monitorate e visionate automaticamente per non permettere aggiornamenti malevoli alla Master Repository

## ASVS - Security Level 1

La seguente sezione di requisiti non funzionali non seguirà il normale ordine stabilito secondo lo standard FURPS, bensì seguirà lo standard definito nella versione v5 di **ASVS** (Application Security Verification Standard), con *jchapter.jsection.jrequirements.j*. Per ognuno di questi requisiti si avrà l'identificatore, secondo lo standard ASVS, descrizione e priorità relativa all'appropriatezza rispetto al dominio d'applicazione di UniClass e all'importanza/interesse rispetto al focus del processo manutentivo, ovvero sviluppo, pipeline e testing finale.

- **1.2.1:** *Verify that output encoding for an HTTP response, HTML document, or XML document is relevant for the context required, such as encoding the relevant characters for HTML elements, HTML attributes, HTML comments, CSS, or HTTP header fields, to avoid changing the message or document structure* - **Priorità Media**
- **1.2.2:** *Verify that when dynamically building URLs, untrusted data is encoded according to its context (e.g., URL encoding or base64url encoding for query or path parameters). Ensure that only safe URL protocols are permitted (e.g., disallow javascript: or data:).* - **Priorità Alta**
- **1.2.3:** *Verify that output encoding or escaping is used when dynamically building JavaScript content (including JSON), to avoid changing the message or document structure (to avoid JavaScript and JSON injection)* - **Priorità Medio-Bassa**
- **1.2.4:** *Verify that data selection or database queries (e.g., SQL, HQL, NoSQL, Cypher) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from SQL Injection and other database injection attacks. This is also relevant when writing stored procedures.* - **Priorità Alta**
- **1.2.5:** *Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding.* - **Priorità Bassa**
- **1.3.1:** *Verify that all untrusted HTML input from WYSIWYG editors or similar is sanitized using a well-known and secure HTML sanitization library or framework feature* - **Priorità Bassa**
- **1.3.2:** *Verify that the application avoids the use of eval() or other dynamic code execution features such as Spring Expression Language (SpEL). Where there is no alternative, any user input being included must be sanitized before being executed.* - **Priorità Alta**
- **1.5.1:** *Verify that the application configures XML parsers to use a restrictive configuration and that unsafe features such as resolving external entities are disabled to prevent XML eXternal Entity (XXE) attacks.* - **Priorità Medio-Bassa**
- **2.1.1:** *Verify that the application's documentation defines input validation rules for how to check the validity of data items against an expected structure. This could be common data formats such as credit card numbers, email addresses, telephone numbers, or it could be an internal data format.* - **Priorità Alta**

- **2.2.1:** *Verify that input is validated to enforce business or functional expectations for that input. This should either use positive validation against an allow list of values, patterns, and ranges, or be based on comparing the input to an expected structure and logical limits according to predefined rules. **For L1, this can focus on input which is used to make specific business or security decisions.** For L2 and up, this should apply to all input. - **Priorità Medio-Alta***
- **2.2.2:** *Verify that the application is designed to enforce input validation at a trusted service layer. While client-side validation improves usability and should be encouraged, it must not be relied upon as a security control. - **Priorità Medio-Alta***
- **2.3.1:** *Verify that the application will only process business logic flows for the same user in the expected sequential step order and without skipping steps. - **Priorità Alta***
- **3.2.1:** *Verify that security controls are in place to prevent browsers from rendering content or functionality in HTTP responses in an incorrect context (e.g., when an API, a user-uploaded file or other resource is requested directly). Possible controls could include: not serving the content unless HTTP request header fields (such as Sec-Fetch-\*) indicate it is the correct context, using the sandbox directive of the Content-Security-Policy header field or using the attachment disposition type in the Content-Disposition header field. - **Priorità Medio-Bassa***
- **3.2.2:** *Verify that content intended to be displayed as text, rather than rendered as HTML, is handled using safe rendering functions (such as createTextNode or textContent) to prevent unintended execution of content such as HTML or JavaScript. - **Priorità Medio-Bassa***
- **3.3.1:** *Verify that cookies have the 'Secure' attribute set, and if the \_\_Host- prefix is not used for the cookie name, the \_\_Secure- prefix must be used for the cookie name. - **Priorità Medio-Bassa***
- **3.4.1:** *Verify that a Strict-Transport-Security header field is included on all responses to enforce an HTTP Strict Transport Security (HSTS) policy. A maximum age of at least 1 year must be defined, and for L2 and up, the policy must apply to all subdomains as well. - **Priorità Bassa***
- **3.4.2:** *Verify that the Cross-Origin Resource Sharing (CORS) Access-Control-Allow-Origin header field is a fixed value by the application, or if the Origin HTTP request header field value is used, it is validated against an allowlist of trusted origins. When 'Access-Control-Allow-Origin: \*' needs to be used, verify that the response does not include any sensitive information. - **Priorità Bassa***
- **3.5.1:** *Verify that, if the application does not rely on the CORS preflight mechanism to prevent disallowed cross-origin requests to use sensitive functionality, these requests are validated to ensure they originate from the application itself. This may be done by using and validating anti-forgery tokens or requiring extra HTTP header fields that are not CORS-safelisted request-header fields. This is to defend against browser-based request forgery attacks, commonly known as cross-site request forgery (CSRF). - **Priorità Bassa***

- **3.5.2:** *Verify that, if the application relies on the CORS preflight mechanism to prevent disallowed cross-origin use of sensitive functionality, it is not possible to call the functionality with a request which does not trigger a CORS-preflight request. This may require checking the values of the 'Origin' and 'Content-Type' request header fields or using an extra header field that is not a CORS-safelisted header-field.* - **Priorità Bassa**
- **3.5.3:** *Verify that HTTP requests to sensitive functionality use appropriate HTTP methods such as POST, PUT, PATCH, or DELETE, and not methods defined by the HTTP specification as "safe" such as HEAD, OPTIONS, or GET. Alternatively, strict validation of the Sec-Fetch-\* request header fields can be used to ensure that the request did not originate from an inappropriate cross-origin call, a navigation request, or a resource load (such as an image source) where this is not expected.* - **Priorità Media**
- **4.1.1:** *Verify that every HTTP response with a message body contains a Content-Type header field that matches the actual content of the response, including the charset parameter to specify safe character encoding (e.g., UTF-8, ISO-8859-1) according to IANA Media Types, such as "text/", "+xml" and "/xml".* - **Priorità Medio-Bassa**
- **4.4.1:** *Verify that WebSocket over TLS (WSS) is used for all WebSocket connections.* - **Priorità Medio-Bassa**
- **5.2.1:** *Verify that the application will only accept files of a size which it can process without causing a loss of performance or a denial of service attack.* - **Priorità Bassa**
- **5.2.2:** *Verify that when the application accepts a file, either on its own or within an archive such as a zip file, it checks if the file extension matches an expected file extension and validates that the contents correspond to the type represented by the extension. This includes, but is not limited to, checking the initial 'magic bytes', performing image re-writing, and using specialized libraries for file content validation. For L1, this can focus just on files which are used to make specific business or security decisions. For L2 and up, this must apply to all files being accepted.* - **Priorità Bassa**
- **5.3.1:** *Verify that files uploaded or generated by untrusted input and stored in a public folder, are not executed as server-side program code when accessed directly with an HTTP request.* - **Priorità Bassa**
- **5.3.2:** *Verify that when the application creates file paths for file operations, instead of user-submitted filenames, it uses internally generated or trusted data, or if user-submitted filenames or file metadata must be used, strict validation and sanitization must be applied. This is to protect against path traversal, local or remote file inclusion (LFI, RFI), and server-side request forgery (SSRF) attacks.* - **Priorità Bassa**

- **6.1.1:** *Verify that application documentation defines how controls such as rate limiting, anti-automation, and adaptive response, are used to defend against attacks such as credential stuffing and password brute force. The documentation must make clear how these controls are configured and prevent malicious account lockout.* - **Priorità Alta**
- **6.2.1:** *Verify that user set passwords are at least 8 characters in length although a minimum of 15 characters is strongly recommended.* - **Priorità Alta**
- **6.2.2:** *Verify that users can change their password.* - **Priorità Media**
- **6.2.3:** *Verify that password change functionality requires the user's current and new password* - **Priorità Media**
- **6.2.4:** *Verify that passwords submitted during account registration or password change are checked against an available set of, at least, the top 3000 passwords which match the application's password policy, e.g. minimum length.* - **Priorità Media**
- **6.2.5:** *Verify that passwords of any composition can be used, without rules limiting the type of characters permitted. There must be no requirement for a minimum number of upper or lower case characters, numbers, or special characters.* - **Priorità Alta**
- **6.2.6:** *Verify that password input fields use type=password to mask the entry. Applications may allow the user to temporarily view the entire masked password, or the last typed character of the password.* - **Priorità Alta**
- **6.2.7:** *Verify that "paste" functionality, browser password helpers, and external password managers are permitted.* - **Priorità Medio-Bassa**
- **6.2.8:** *Verify that the application verifies the user's password exactly as received from the user, without any modifications such as truncation or case transformation.* - **Priorità Alta**
- **6.3.1:** *Verify that controls to prevent attacks such as credential stuffing and password brute force are implemented according to the application's security documentation.* - **Priorità Alta**
- **6.3.2:** *Verify that default user accounts (e.g., "root", "admin", or "sa") are not present in the application or are disabled.* - **Priorità Media**
- **6.4.1:** *Verify that system generated initial passwords or activation codes are securely randomly generated, follow the existing password policy, and expire after a short period of time or after they are initially used. These initial secrets must not be permitted to become the long term password.* - **Priorità Bassa**
- **6.4.2:** *Verify that password hints or knowledge-based authentication (so-called "secret questions") are not present.* - **Priorità Medio-Alta**
- **7.2.1:** *Verify that the application performs all session token verification using a trusted, backend service.* - **Priorità Alta**

- **7.2.2:** *Verify that the application uses either self-contained or reference tokens that are dynamically generated for session management, i.e. not using static API secrets and keys* - **Priorità Medio-Alta**
- **7.2.3:** *Verify that if reference tokens are used to represent user sessions, they are unique and generated using a cryptographically secure pseudo-random number generator (CSPRNG) and possess at least 128 bits of entropy.* - **Priorità Medio-Bassa**
- **7.2.4:** *Verify that the application generates a new session token on user authentication, including re-authentication, and terminates the current session token.* - **Priorità Media**
- **7.4.1:** *Verify that when session termination is triggered (such as logout or expiration), the application disallows any further use of the session. For reference tokens or stateful sessions, this means invalidating the session data at the application backend. Applications using self-contained tokens will need a solution such as maintaining a list of terminated tokens, disallowing tokens produced before a per-user date and time or rotating a per-user signing key.* - **Priorità Alta**
- **7.4.2:** *Verify that the application terminates all active sessions when a user account is disabled or deleted (such as an employee leaving the company)* - **Priorità Alta**
- **8.1.1:** *Verify that authorization documentation defines rules for restricting function-level and data-specific access based on consumer permissions and resource attributes.* - **Priorità Alta**
- **8.2.1:** *Verify that the application ensures that function-level access is restricted to consumers with explicit permissions.* **Priorità Media**
- **8.2.2:** *Verify that the application ensures that data-specific access is restricted to consumers with explicit permissions to specific data items to mitigate insecure direct object reference (IDOR) and broken object level authorization (BOLA).* - **Priorità Bassa**
- **8.3.1:** *Verify that the application enforces authorization rules at a trusted service layer and doesn't rely on controls that an untrusted consumer could manipulate, such as client-side JavaScript.* - **Priorità Alta**
- **9.1.1:** *Verify that self-contained tokens are validated using their digital signature or MAC to protect against tampering before accepting the token's contents* - **Priorità Medio-Bassa**
- **9.1.2:** *Verify that only algorithms on an allowlist can be used to create and verify self-contained tokens, for a given context. The allowlist must include the permitted algorithms, ideally only either symmetric or asymmetric algorithms, and must not include the 'None' algorithm. If both symmetric and asymmetric must be supported, additional controls will be needed to prevent key confusion* - **Priorità Media**

- **9.1.3:** *Verify that key material that is used to validate self-contained tokens is from trusted pre-configured sources for the token issuer, preventing attackers from specifying untrusted sources and keys. For JWTs and other JWS structures, headers such as 'jku', 'x5u', and 'jwk' must be validated against an allowlist of trusted sources. - Priorità Bassa*
- **9.1.4:** *Verify that, if a validity time span is present in the token data, the token and its content are accepted only if the verification time is within this validity time span. For example, for JWTs, the claims 'nbf' and 'exp' must be verified. - Priorità Medio-Bassa*
- **10.4.1:** *Verify that the authorization server validates redirect URIs based on a client-specific allowlist of pre-registered URIs using exact string comparison. - Priorità Media*
- **10.4.2:** *Verify that, if the authorization server returns the authorization code in the authorization response, it can be used only once for a token request. For the second valid request with an authorization code that has already been used to issue an access token, the authorization server must reject a token request and revoke any issued tokens related to the authorization code. - Priorità Media*
- **10.4.3:** *Verify that the authorization code is short-lived. The maximum lifetime can be up to 10 minutes for L1 and L2 applications and up to 1 minute for L3 applications. - Priorità Media*
- **10.4.4:** *Verify that for a given client, the authorization server only allows the usage of grants that this client needs to use. Note that the grants 'token' (Implicit flow) and 'password' (Resource Owner Password Credentials flow) must no longer be used. - Priorità Medio-Bassa*
- **10.4.5:** *Verify that the authorization server mitigates refresh token replay attacks for public clients, preferably using sender-constrained refresh tokens, i.e., Demonstrating Proof of Possession (DPoP) or Certificate-Bound Access Tokens using mutual TLS (mTLS). For L1 and L2 applications, refresh token rotation may be used. If refresh token rotation is used, the authorization server must invalidate the refresh token after usage, and revoke all refresh tokens for that authorization if an already used and invalidated refresh token is provided - Priorità Medio-Bassa*
- **11.1.3:** *Verify that insecure block modes (e.g., ECB) and weak padding schemes (e.g., PKCS#1 v1.5) are not used. - Priorità Alta*
- **11.1.4:** *Verify that only approved ciphers and modes such as AES with GCM are used. - Priorità Alta*
- **11.4.1:** *Verify that only approved hash functions are used for general cryptographic use cases, including digital signatures, HMAC, KDF, and random bit generation. Disallowed hash functions, such as MD5, must not be used for any cryptographic purpose. - Priorità Media*

- **12.1.1:** *Verify that only the latest recommended versions of the TLS protocol are enabled, such as TLS 1.2 and TLS 1.3. The latest version of the TLS protocol must be the preferred option.* - **Priorità Medio-Bassa**
- **12.2.1:** *Verify that TLS is used for all connectivity between a client and external facing, HTTP-based services, and does not fall back to insecure or unencrypted communications.* - **Priorità Medio-Bassa**
- **12.2.2:** *Verify that external facing services use publicly trusted TLS certificates.* - **Priorità Medio-Bassa**
- **13.4.1:** *Verify that the application is deployed either without any source control metadata, including the .git or .svn folders, or in a way that these folders are inaccessible both externally and to the application itself.* - **Priorità Alta**
- **14.3.1:** *Verify that authenticated data is cleared from client storage, such as the browser DOM, after the client or session is terminated. The 'Clear-Site-Data' HTTP response header field may be able to help with this but the client-side should also be able to clear up if the server connection is not available when the session is terminated* - **Priorità Alta**
- **15.1.1:** *Verify that application documentation defines risk based remediation time frames for 3rd party component versions with vulnerabilities and for updating libraries in general, to minimize the risk from these components* - **Priorità Alta**
- **15.2.1:** *Verify that the application only contains components which have not breached the documented update and remediation time frames* - **Priorità Alta**
- **15.3.1:** *Verify that the application only returns the required subset of fields from a data object. For example, it should not return an entire data object, as some individual fields should not be accessible to users.* - **Priorità Alta**

## 3.5 Modelli di Sistema

### 3.5.1 Scenari

Sono mostrati gli scenari e casi d'uso che rappresentano le funzionalità più complesse e di rilievo della piattaforma

**Scenario 1: Autenticazione** L'utente AnnaMaria Franceschi vuole utilizzare la piattaforma web UniClass con l'utilizzo delle credenziali da poco ottenute. AnnaMaria apre la piattaforma web UniClass e una volta sulla schermata principale del sito, clicca in alto a destra sull'icona rappresentante un omino. A quel punto viene presentato un form in cui deve inserire la sua email e la sua password. Una volta inserite le sue credenziali di accesso univoche, con l'uso del tasto "Login" viene effettuata l'autenticazione e viene riportata sulla pagina principale. A questo punto AnnaMaria visualizza i servizi offerti agli utenti UniClass.

**Scenario 2: Visualizzare l'orario** Lo studente Marco ha bisogno di visualizzare l'orario settimanale del suo corso di studi. Marco, quindi, decide di accedere alla piattaforma web UniClass. Sfortunatamente, non ricorda le credenziali per accedere all'applicazione web, ma questo non è un problema, poiché l'applicazione presenta nella homepage una sezione contenente un form. Il form contiene tre campi di selezione: corso di studi, Anno di studi e resto. Essendo uno studente del terzo anno d'informatica all'Università degli Studi di Salerno, del resto 0, seleziona nel primo campo l'opzione "Informatica", nel secondo "Terzo", e nel terzo campo l'opzione "0". Al completamento del Form, Marco clicca il tasto "Mostra Orario", che permette di confermare la sua scelta e, finalmente, visualizza l'orario delle sue lezioni.

**Scenario 3: Visualizzare Aule libere** La studentessa Carlotta ha appena terminato la lezione di Ingegneria del Software alle 13:00 e sta cercando un'aula in cui poter sistemare gli appunti presi a lezione. Carlotta non sa quali siano le aule libere in questo momento, quindi decide di aprire il sito web di UniClass. Carlotta seleziona nel menù a tendina a sinistra l'opzione "Aule Libere" e, una volta scelto l'edificio più vicino a lei tra tutti quelli presenti, lo seleziona. Carlotta visualizza quindi una lista contenente le aule dell'edificio, seleziona l'aula desiderata e visualizza gli orari in cui l'aula è occupata oppure libera. Nota che l'aula F8 nell'edificio F2 è disponibile, ma solo fino alle 14:00, e selezionando poi l'aula F6 vede che risulta disponibile dalle 14:30. Carlotta riesce ad organizzare la giornata di studi, presentandosi prima in F8 e poi in F6.

**Scenario 4: Messaggistica da parte dello studente** La studentessa Mara Giuditti di Beni Culturali dell'Università degli studi di Salerno ha bisogno di comunicare alla docente Lidia Tornatore che non potrà più recarsi al suo ufficio per discutere di un suo progetto a causa di problemi di mezzi di trasporto pubblici. Mara apre la piattaforma UniClass, accede tramite le credenziali fornite (e-mail universitaria univoca e password) e, attraverso il menu a tendina, sceglie l'opzione "Messaggistica". Da lì cerca nella lista dei docenti Lidia Tornatore. Dopo aver effettuato la sua selezione, si apre la chat e da lì scrive il messaggio che desidera comunicare.

**Scenario 5: Messaggistica da parte del docente** La docente Lidia Tornatore è in attesa dell'arrivo della studentessa Mara Giuditti. La docente, che stava controllando quante persone fossero presenti stamattina a lezione. Dalla pagina in cui si trova sulla piattaforma UniClass, clicca il menu a tendina in alto a sinistra e seleziona la voce "Messaggistica". Da lì, la docente visualizza il messaggio completo dall'unica chat al momento non letta. Scrive il messaggio di risposta e lo invia.

**Scenario 6: Messaggistica Broadcast (Professore)** La professoressa Rosanna Ventura del corso di Inglese dell'Università di Salerno ha bisogno di comunicare ai suoi studenti che domani mattina non sarà presente a lezione a causa di imprevisti. Per avvisare i suoi alunni, la professoressa decide di aprire la piattaforma UniClass, autenticarsi con le sue credenziali (e-mail universitaria univoca e password) nella schermata di Log In accessibile dal pulsante situato in alto a destra della pagina e accedere alla voce "Messaggistica" nel menù a tendina in alto a sinistra. Nella schermata di messaggistica, in alto a sinistra, la professoressa visualizza l'opzione "Topic" e, dopo averla selezionata, scrive

il topic interessato alla variazione dell'orario della lezione. Invia l'avviso e, dopo l'invio dell'avviso, gli studenti verranno notificati.

**Scenario 7: Visualizzare informazioni sul proprio profilo** Lo studente Giancarlo Esposito non ricorda la sua matricola, necessaria per la partecipazione all'esame nell'aula F8 dell'edificio F2. Rapidamente, decide di aprire la piattaforma di fiducia UniClass. Una volta sulla homepage del sito, clicca il pulsante per accedere in alto a destra. Nella schermata di Log In, inserisce le sue credenziali (e-mail universitaria univoca e password) ed accede finalmente alla pagina web. A questo punto, seleziona nuovamente lo stesso pulsante dell'autenticazione che, stavolta, lo indirizza a una pagina contenente le informazioni sul suo profilo d'ateneo. Giancarlo visualizza la sua immagine profilo ed i propri dati anagrafici secondo questo ordine: "Nome", "Cognome", "email", "Matricola", "Corso di Laurea". Dopo aver visualizzato la matricola, è pronto a firmare il documento di partecipazione all'esame con la sua matricola.

**Scenario 8: Messaggistica broadcast (Coordinatore di Corso)** Il Coordinatore del corso di Informatica dell'Università degli Studi di Salerno, Fabio Fabrizi, deve comunicare la data del nuovo seminario con un'azienda IoT internazionale a tutti gli studenti. Per fare ciò, apre il sito web di UniClass e si autentica tramite un click sul pulsante posizionato in alto a destra nell'homepage e inserisce le sue credenziali (e-mail universitaria univoca e password). Dopo l'autenticazione, Fabio Fabrizi preme il pulsante del menù a tendina posto in alto a sinistra e seleziona la voce "Messaggistica". Nella pagina dei messaggi, seleziona la voce "Topic" con gli studenti come destinatari dell'avviso. Fabio Fabrizi scrive il suo avviso e lo invia. Il messaggio è stato inviato con successo a tutti.

**Scenario 9: Creazione dell'account UniClass** Giovanni, personale tecnico-amministrativo, ha appena ricevuto i dati degli studenti immatricolati, che hanno superato il TOLC del 4 settembre. Ad ogni studente, come consuetudine, deve essere assegnato un account UniClass. Giovanni dovrà attivare questo account. Giovanni accede come personale tecnico-amministrativo alla piattaforma UniClass. Una volta completato l'accesso, apre il menù a tendina a sinistra e seleziona la voce "Aggiungi Account". Compila il form inserendo e-mail universitaria, il ruolo dell'account e verrà creato l'account di UniClass con l'email universitaria assegnata nel form ed una password generata randomicamente, da inviare allo studente.

**Scenario 10: Rimozione dell'Account** Alfonso del personale tecnico-amministrativo della piattaforma UniClass riceve, tramite una e-mail, una richiesta di eliminazione di un account di tipo "Studente" a causa della sua rinuncia agli studi. Alfonso accederà con le sue credenziali come personale tecnico-amministrativo sulla piattaforma UniClass. Dopo aver effettuato l'accesso, dalla homepage aprirà il menù a tendina a sinistra e sceglierà la voce "Rimuovi Account". Compila il Form digitando l'e-mail associata allo studente e seleziona l'opzione "Rimuovi". L'account sarà rimosso.

### 3.5.2 Use Case Model

Prima di visualizzare i vari casi d'uso riguardanti gli scenari descritti in precedenza, studiamo la gerarchia degli utenti presente all'interno del sistema UniClass.



#### Scenario 1 – Autenticazione (UC1) Caso d'uso: autentica

**Attore:** Ospite

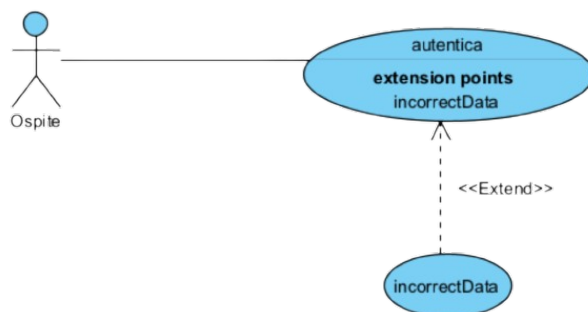
**Precondizione:** l'utente non è autenticato

**Flusso di eventi:**

1. L'ospite clicca in alto a destra l'icona del profilo
2. L'ospite inserisce l'email e password del suo profilo
3. L'ospite è finalmente autenticato

#### Flussi Alternativi:

2.1) L'utente visualizza un messaggio d'errore per la mancata corrispondenza tra email e password nel database.



#### Scenario 2 – Visualizzare l'Orario (UC2) Caso d'uso: visualizzaOrario

**Attore:** Ospite

**Precondizioni:** Il caso d'uso inizia quando l'utente naviga la piattaforma per visualizzare

l'orario del proprio corso

**Flusso di eventi:**

1. L'utente apre il menù a tendina a sinistra
2. L'utente seleziona la voce "Orari"
3. L'utente compila il Form presente nella parte superiore della pagina, inserendo corso di laurea, resto/sezione e anno

**Postcondizione:** Il caso d'uso termina quando l'utente visualizza finalmente l'orario, dopo aver confermato la compilazione del Form.

**Scenario 3 – Visualizzare Aule Libere (UC3) Caso d'uso: visualizzaAule**

**Attore:** Ospite

**Precondizioni:** il caso d'uso inizia quando l'utente naviga la piattaforma per visualizzare le aule libere disponibili

**Flusso di eventi:**

1. l'utente apre il menù a tendina a sinistra
2. l'utente seleziona la voce "Aule Libere"
3. l'utente seleziona l'edificio desiderato
4. l'utente visualizza le aule dell'edificio

**Postcondizione:** il caso d'uso termina quando l'utente visualizza finalmente le aule libere dell'edificio desiderato con gli orari di disponibilità

**Scenario 4 – Messaggistica da parte dello Studente (UC4) Caso d'uso: messaggioStudente**

**Attore:** Studente

**Precondizione:** l'utente è autenticato come Studente

**Flusso di eventi:**

1. lo studente apre il menù a tendina a sinistra

2. lo studente seleziona la voce "Messaggistica"
3. lo studente seleziona il docente con il quale vuole comunicare
4. lo studente scrive il messaggio da inviare al docente selezionato

**Postcondizione:** Il caso d'uso termina quando lo studente visualizza nella chat il messaggio inviato al docente



**Scenario 5 – Messaggistica da parte del Docente (UC5)** Caso d'uso: messaggioDocente

**Attore:** Docente

**Precondizione:** l'utente è autenticato come Docente

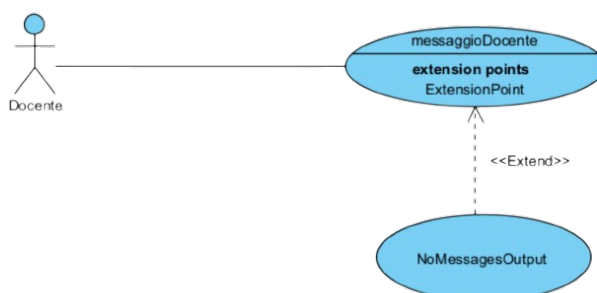
**Flusso di eventi:**

1. il docente apre il menù a tendina a sinistra
2. il docente seleziona la voce "Messaggistica"
3. il docente seleziona l'anteprima del profilo di uno studente in particolare
4. il docente invia un messaggio allo studente selezionato

**Postcondizione:** Il caso d'uso termina quando il docente risponde al messaggio dello studente selezionato

**Flusso Alternativo:**

(3) In caso di mancati messaggi, Il docente visualizza la lista vuota



**Scenario 6 – Messaggistica Broadcast (Professore) (UC6)** Caso d'uso: messaggioBroadcast

**Attore:** Docente

**Precondizione:** l'utente è autenticato come Docente

**Flusso di eventi:**

1. Il docente apre il menù a tendina a sinistra
2. Il docente seleziona la voce "Messaggistica"
3. Il docente seleziona l'opzione "Topic"
4. Il docente scrive il topic interessato al messaggio
5. Il docente scrive l'avviso

**Postcondizione:** Il caso d'uso termina quando il docente comunica l'avviso riguardante l'esame selezionato e gli studenti ricevono il messaggio.



**Scenario 7 – Visualizzare informazioni sul proprio profilo (UC7) Caso d'uso:** visualizzaProfilo

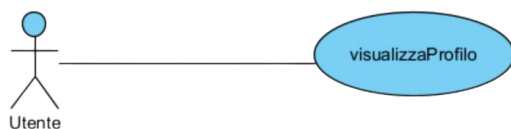
**Attore:** Utente

**Precondizione:** l'utente è autenticato nella piattaforma

**Flusso di eventi:**

1. L'utente visualizza l'homepage
2. L'utente seleziona l'icona del profilo in alto a destra
3. L'utente visualizzerà tutte le informazioni del suo profilo

**Postcondizione:** Il caso d'uso termina quando il coordinatore invia la comunicazione a tutti gli studenti del corso



**Scenario 8 – Messaggistica Broadcast (Coordinatore) (UC8) Caso d'uso:** messaggioCoordinatore

**Attore:** Coordinatore del corso

**Precondizione:** l'utente è autenticato come Coordinatore di corso

**Flusso di eventi:**

1. il coordinatore apre il menù a tendina a sinistra
2. il coordinatore seleziona la voce "Messaggistica"

3. il coordinatore visualizza le anteprime degli accademici con cui sono avvenuti messaggi recenti
4. il coordinatore invia un messaggio per tutti gli studenti del corso con il "Topic" non vuoto

**Postcondizione:** Il caso d'uso termina quando il coordinatore invia la comunicazione a tutti gli studenti del corso



**Caso d'uso:** creaAccountUniClass **Attore:** Ospite **Precondizione:** L'utente non è autenticato nel sistema.

**Flusso di eventi:**

1. L'utente accede alla sezione "Gestione Utenti".
2. L'utente seleziona l'opzione "Crea Nuovo Account".
3. L'utente compila il form inserendo i dati anagrafici, l'email istituzionale e seleziona il **Ruolo** da assegnare al nuovo utente (es. Studente, Docente).
4. Il sistema verifica la validità dei dati e l'unicità dell'email.
5. Il sistema genera automaticamente una password temporanea.
6. Il sistema crea l'entità **Utente** e le associa il Ruolo specificato.
7. Il sistema invia le credenziali all'utente tramite email.

**Flussi Alternativi:**

- (3.a) Email già esistente: Il sistema notifica l'errore e suggerisce il recupero password.
- (4.a) Dati non validi: Il sistema evidenzia i campi errati.

**Postcondizione:** Viene creato un nuovo Utente nel sistema con uno specifico Ruolo associato e credenziali di accesso valide.

**Scenario 9 - Crea/Attiva Account (PersonaleTA) (UC9)** **Caso d'uso:** aggiungiAccount

**Attore:** PersonaleTA

**Precondizione:** L'utente è autenticato come Personale Tecnico-Amministrativo

**Flusso di Eventi:**

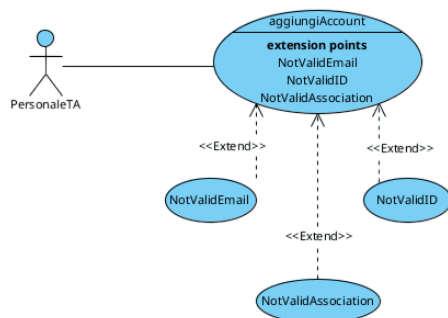
1. Il personale tecnico-amministrativo apre il menù a tendina a sinistra dalla Homepage

2. Il personale tecnico-amministrativo seleziona la voce "Utenti"
3. Il personale tecnico-amministrativo visualizza l'utente desiderato per l'operazione di creazione account
4. Il personale tecnico-amministrativo inserisce le informazioni dell'utente e seleziona "Attiva Account"

**Postcondizione:** Il caso d'uso termina quando il Personale Tecnico Amministrativo attiva l'utente selezionato e quest'ultimo può finalmente autenticarsi nella piattaforma.

**Flusso Alternativo:**

- (4.a) In caso di inserimento dell'email sbagliata, visualizzerà un errore.
- (4.b) In caso di inserimento della matricola sbagliata, visualizzerà un errore
- (4.c) In caso di associazione non valida tra le credenziali inserite, visualizzerà un errore



**Scenario 10 - Rimuovi/Disabilita (PersonaleTA) (UC10) Caso d'uso:** rimuoviAccount

**Attore:** PersonaleTA

**Precondizione:** L'utente è autenticato come Personale Tecnico-Amministrativo

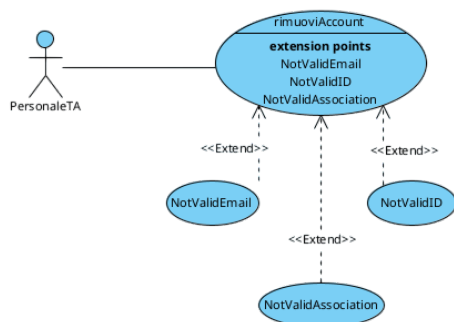
**Flusso di Eventi:**

1. Il personale tecnico-amministrativo apre il menù a tendina a sinistra dalla Homepage
2. Il personale tecnico-amministrativo seleziona la voce "Utenti"
3. Il personale tecnico-amministrativo visualizza l'utente desiderato per l'operazione di rimozione account
4. Il personale tecnico-amministrativo inserisce le informazioni dell'utente e seleziona "Rimuovi Account"

**Postcondizione:** Il caso d'uso termina quando il Personale Tecnico Amministrativo rimuove l'account selezionato e quest'ultimo non può più autenticarsi nella piattaforma (con le credenziali originarie). **Flusso Alternativo:** (4.a) In caso di inserimento dell'email sbagliata, visualizzerà un errore.

(4.b) In caso di inserimento della matricola sbagliata, visualizzerà un errore

(4.c) In caso di associazione non valida tra le credenziali inserite, visualizzerà un errore

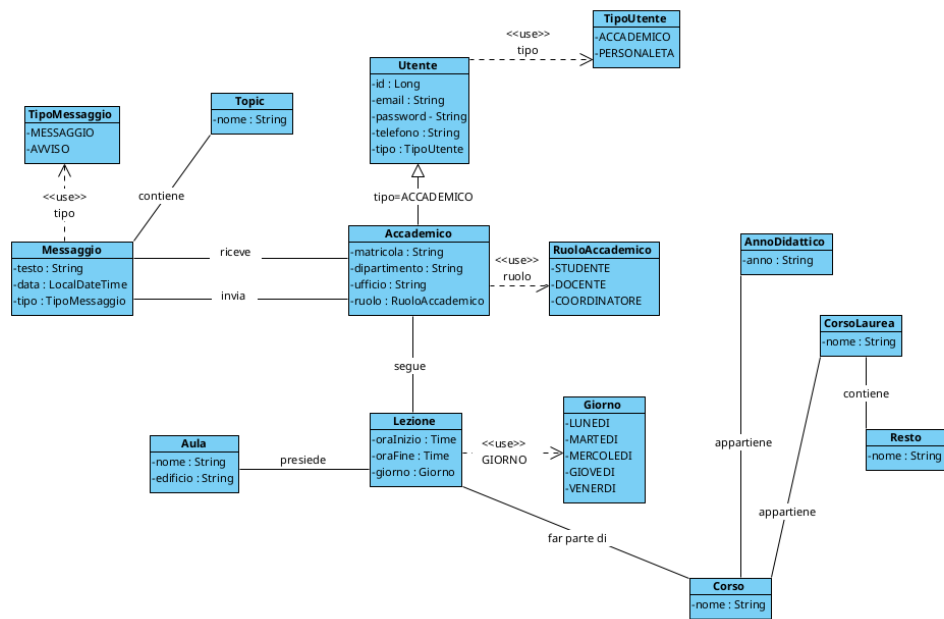


### 3.5.3 Object Model

Per la creazione dell'Object Model, si è deciso di creare package in base alle caratteristiche da evidenziare, come la rappresentazione delle varietà degli utenti del sistema (vedi la Prima Illustrazione di UML nel Modello degli Oggetti). Gli **utenti** hanno attributi e metodi diversi e possono essere classificati in due gruppi principali, che sono il personale e l'accademico, per distinguere gli utenti dagli interni o esterni all'esperienza accademica. La classe 'Accademico' figura solo nel package 'Utenti'. Anche se questa classe non è rappresentata in tutti gli altri pacchetti, le varie proprietà della suddetta classe sono state replicate in tutte le sue sottoclassi. Questo non significa che la classe 'Accademico' sia completamente assente dagli altri pacchetti, ma piuttosto, per evitare di perdere concentrazione sulle classi base di ciascun pacchetto, si è deciso di non introdurla.

Per il package "**Messaggistica**" vari utenti possono inviare o ricevere messaggi. Ogni messaggio può avere un tipo ("Annuncio", "Messaggio") ed il tipo "Annuncio" comprende un oggetto.

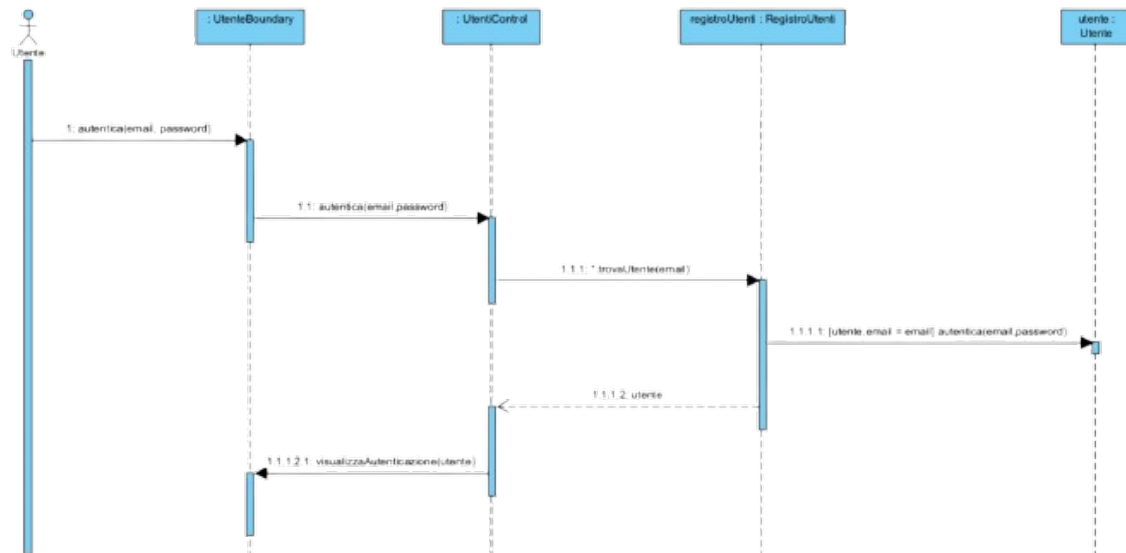
Il package "**OrarioAule**" è importante per capire la classe "Orario", poiché ogni lezione fa parte di un orario ed è localizzata in Aula. Ogni aula può avere lezioni o meno e ciò sarà cruciale per capire la disponibilità, ora per ora, di un'aula.



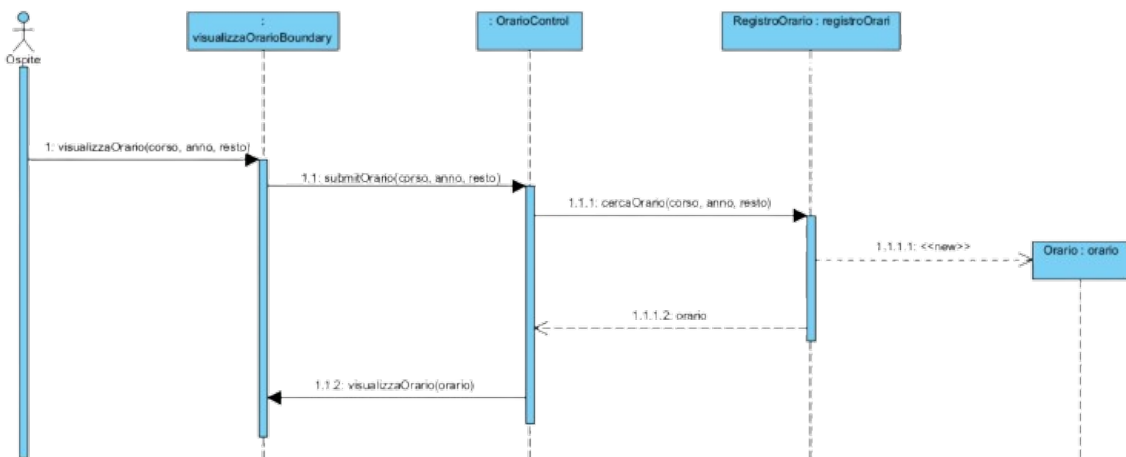
### 3.5.4 Dynamic Model

#### Sequence Diagram

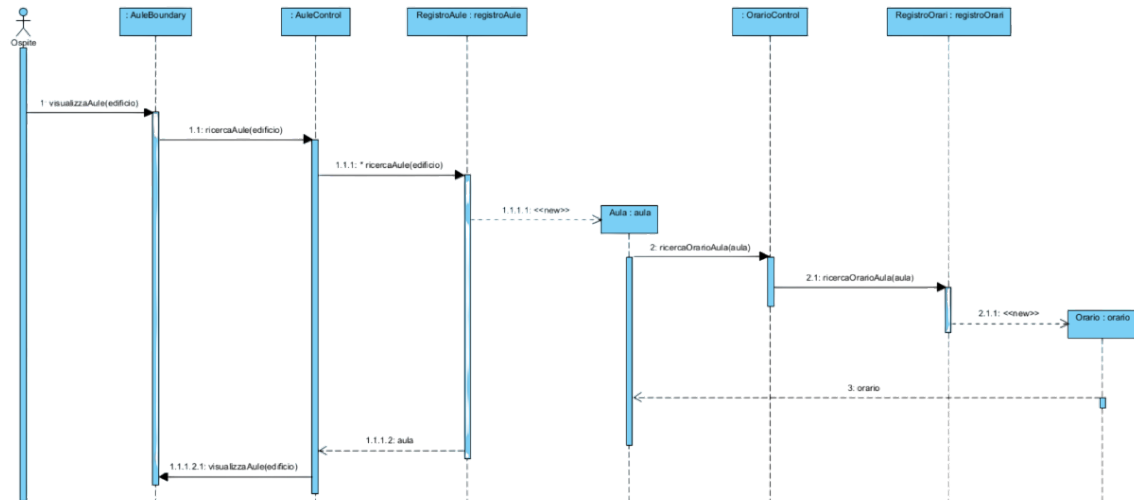
##### Scenario 1 – Autenticazione (UC1)



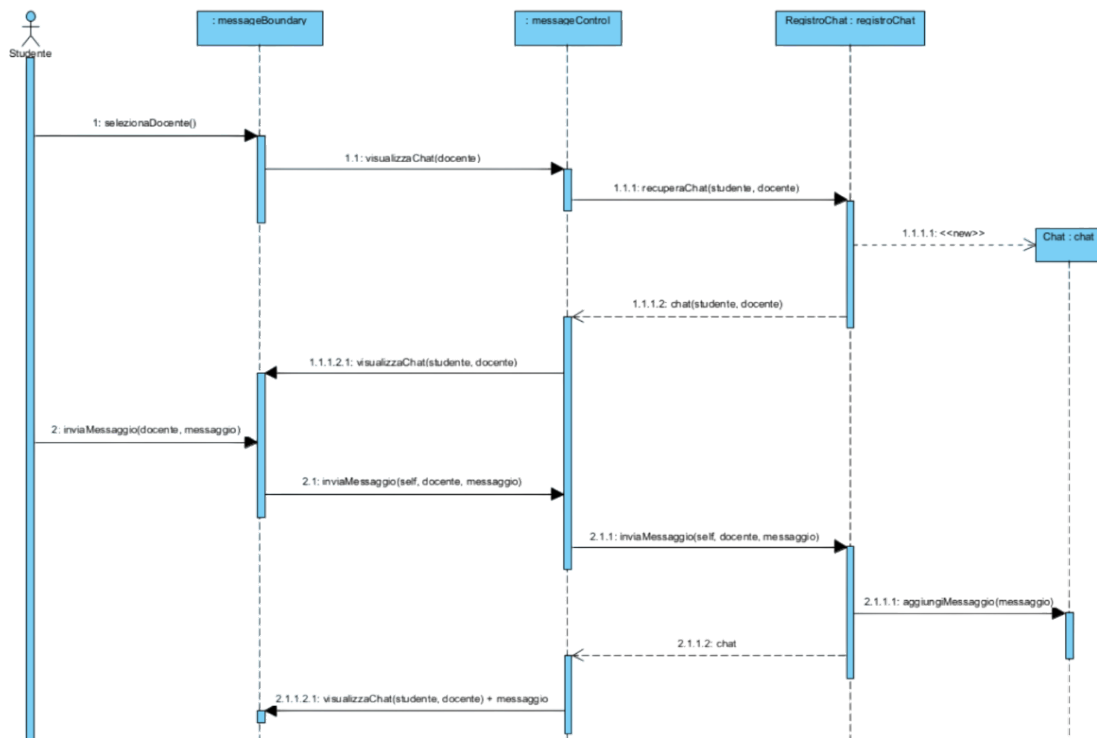
##### Scenario 2 – Visualizzare l'Orario (UC2)



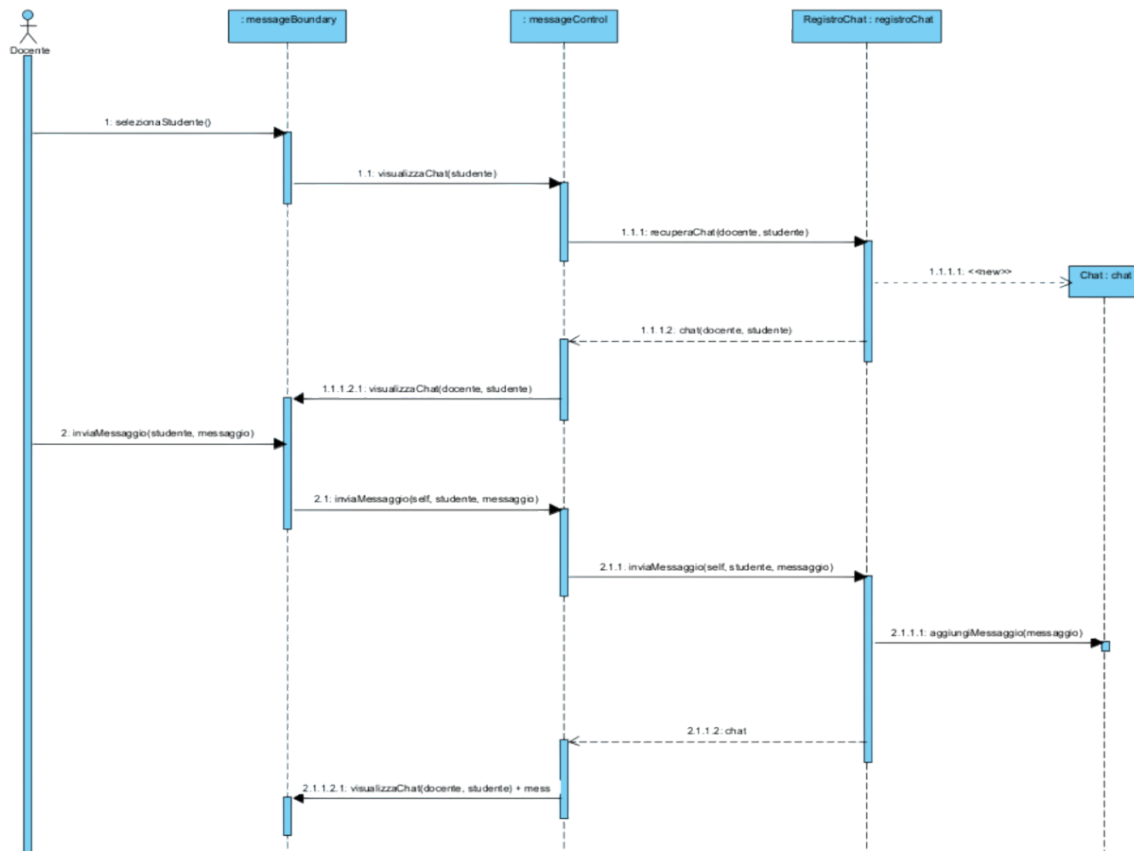
### Scenario 3 – Visualizza Aule Libere (UC3)



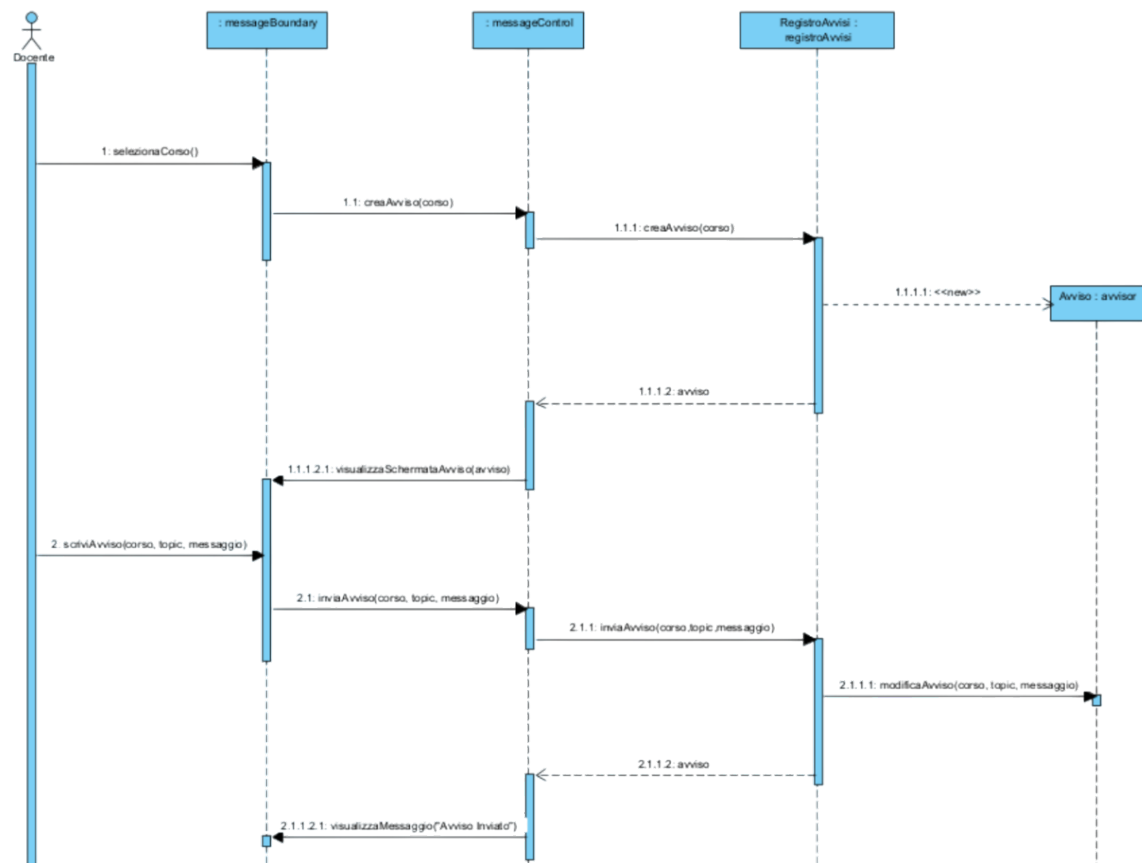
### Scenario 4 – Messaggio inviato dallo Studente (UC4)



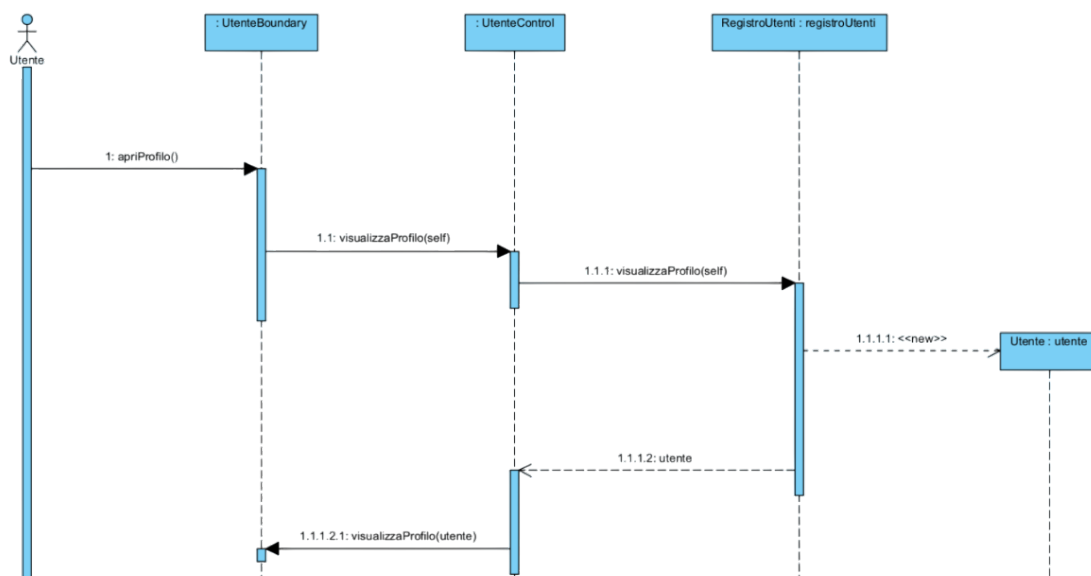
### Scenario 5 – Messaggistica da parte del Docente (UC5)



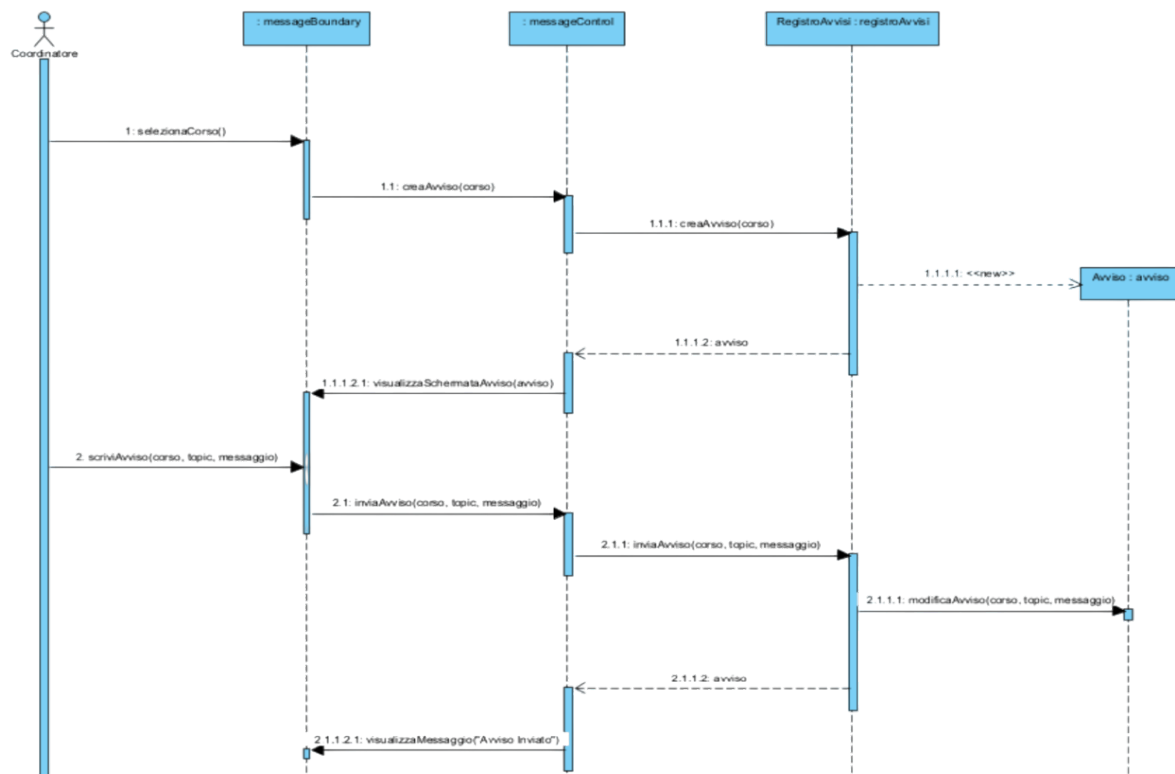
### Scenario 6 – Messaggio Broadcast del Docente (UC6)



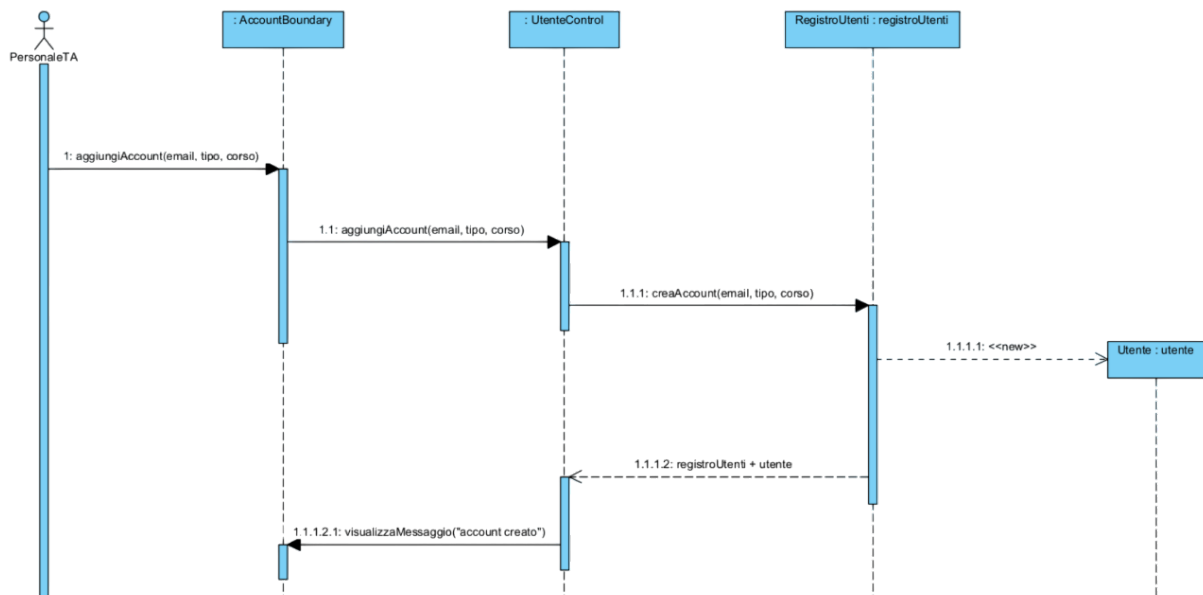
### Scenario 7 – Visualizza informazioni Profilo (UC7)



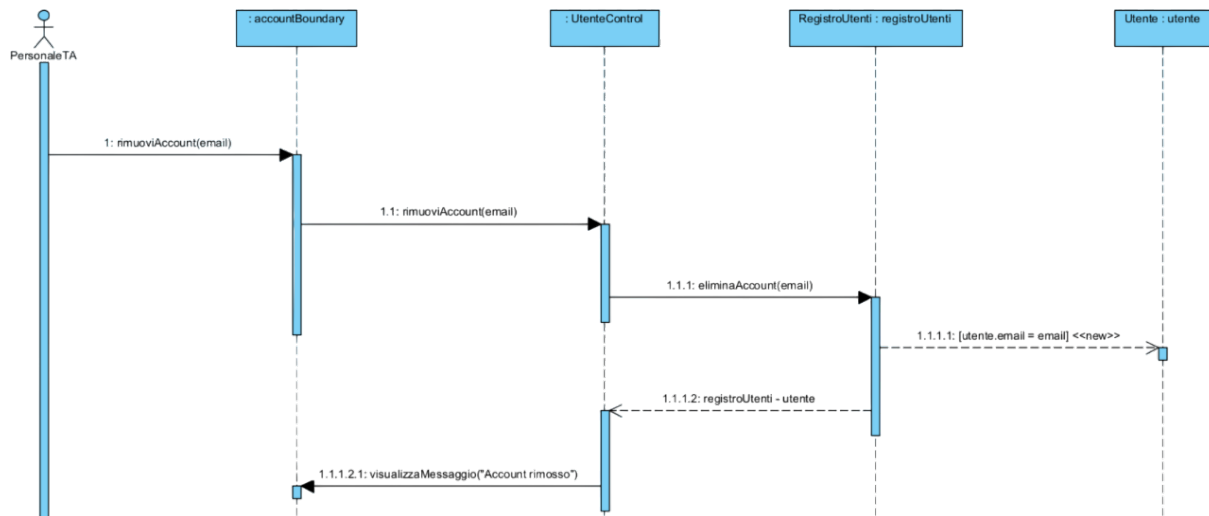
### Scenario 8 – Messaggistica Broadcast (Coordinatore) (UC8)



### Scenario 9 – Creazione Account UniClass (UC9)



### Scenario 10 - Rimozione Account UniClass (UC10)



## 4 AI-Usage

È stata richiesta l'assistenza dell'IA **solo ed esclusivamente** per la formattazione della tabella. Tutto ciò che è documentato è frutto della creatività e dell'esperienza del sottoscritto.